

词典

散列：循对象访问

联合数组：更直接、更有效的访问

❖ 数组？再常见不过，比如：

`fib[0] = 0`

`fib[1] = 1`

`fib[2] = 1`

`fib[3] = 2`

`fib[4] = 3`

`fib[5] = 5`

`fib[6] = 8`

...

❖ Associative Array

——与此前的数组有何区别？

❖ 根据数据元素的取值，直接访问！

`style["关羽"] = "云长"`

`style["张飞"] = "翼德"`

`style["赵云"] = "子龙"`

`style["马超"] = "孟起"`

❖ 下标不再是**整数**，甚至没有大小**次序**

——更为直观、便捷

❖ 支持的语言：

Snobol4、MUMPS、SETL、Rexx、AWK、

Java、Python、Perl、Ruby、PHP、...

词条 ~ 映射/词典

❖ entry = (key, value)

❖ Map/Dictionary: 词条的集合

- 关键码禁止/允许雷同

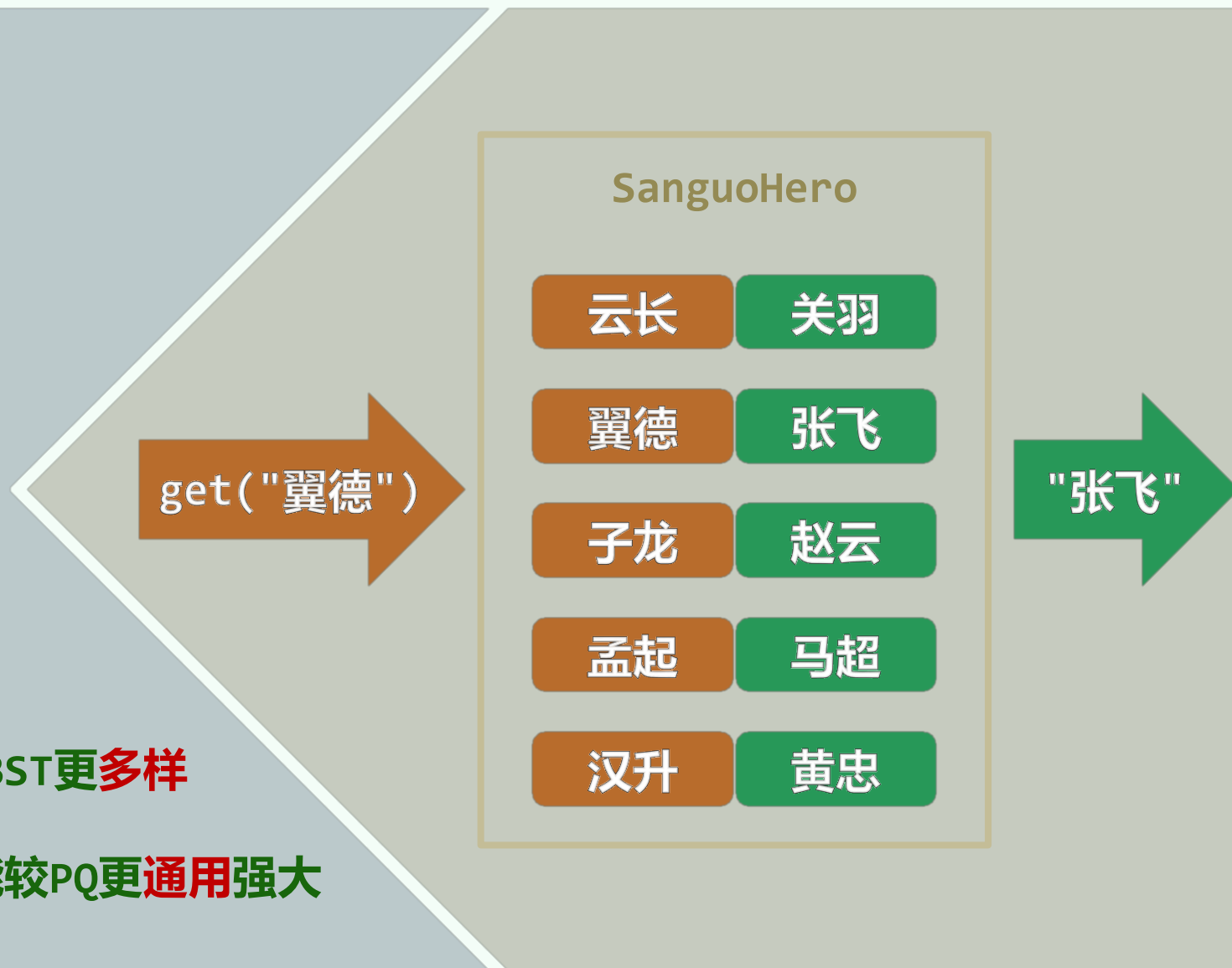
- get(key)

put(key, value)

remove(key)

❖ 关键码不必可比较大小, 元素类型较BST更多样

查找对象不限于最大/最小词条, 功能较PQ更通用强大



词典
散列： 原理

电话: 号码 ~ 人

General inquiries

Tel: Toll Free: 1-800-IBM-4YOU

E-mail: askibm@vnet.ibm.com

www.ibm.com/us/en/

Shopping

Tel: Toll Free: 1-888-SHOP-IBM

Sales Center

1-855-2-LENOVO (1-855-253-6686)

Mon - Fri: 9am-9pm (EST)

Sat - Sun: 9am-6pm (EST)

Customer Service

1-855-2-LENOVO (1-855-253-6686)

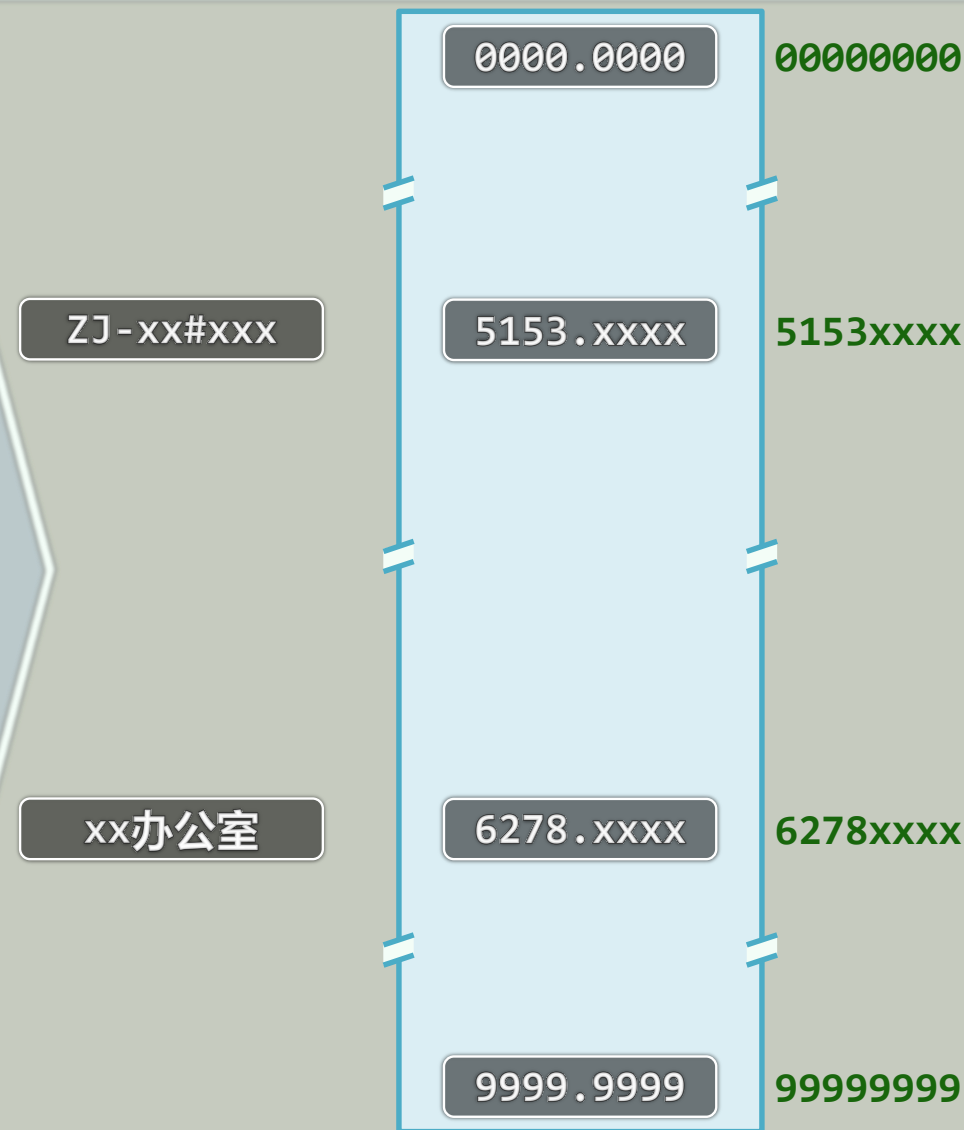
Mon - Fri: 9am-9pm (EST)

Sat - Sun: 9am-6pm (EST)



电话簿

- ❖ 需求：为一所学校制作电话簿
号码 --> 教员、学生、员工、办公室
- ❖ 蛮力：用户记录 ~ 数组Rank ~ 电话号码
 $O(1)$ 效率!
- ❖ 以清华为例 (2003)
#可能的电话 = $\mathcal{R} = 10^8 = 100\text{M}$
#实有的电话 = $\mathcal{N} = 25,000 = 25\text{K}$
- ❖ 问题：空间 = $O(\mathcal{R} + \mathcal{N}) = O(100\text{M} + 25\text{K})$
效率 = $25\text{K} / 100\text{M} = 0.025\%$
- ❖ 如何在保持查找速度的同时，降低存储消耗?



散列表 / 散列函数

❖ 桶 (bucket) : 直接存放或间接指向一个词条

❖ Bucket array ~ Hashtable

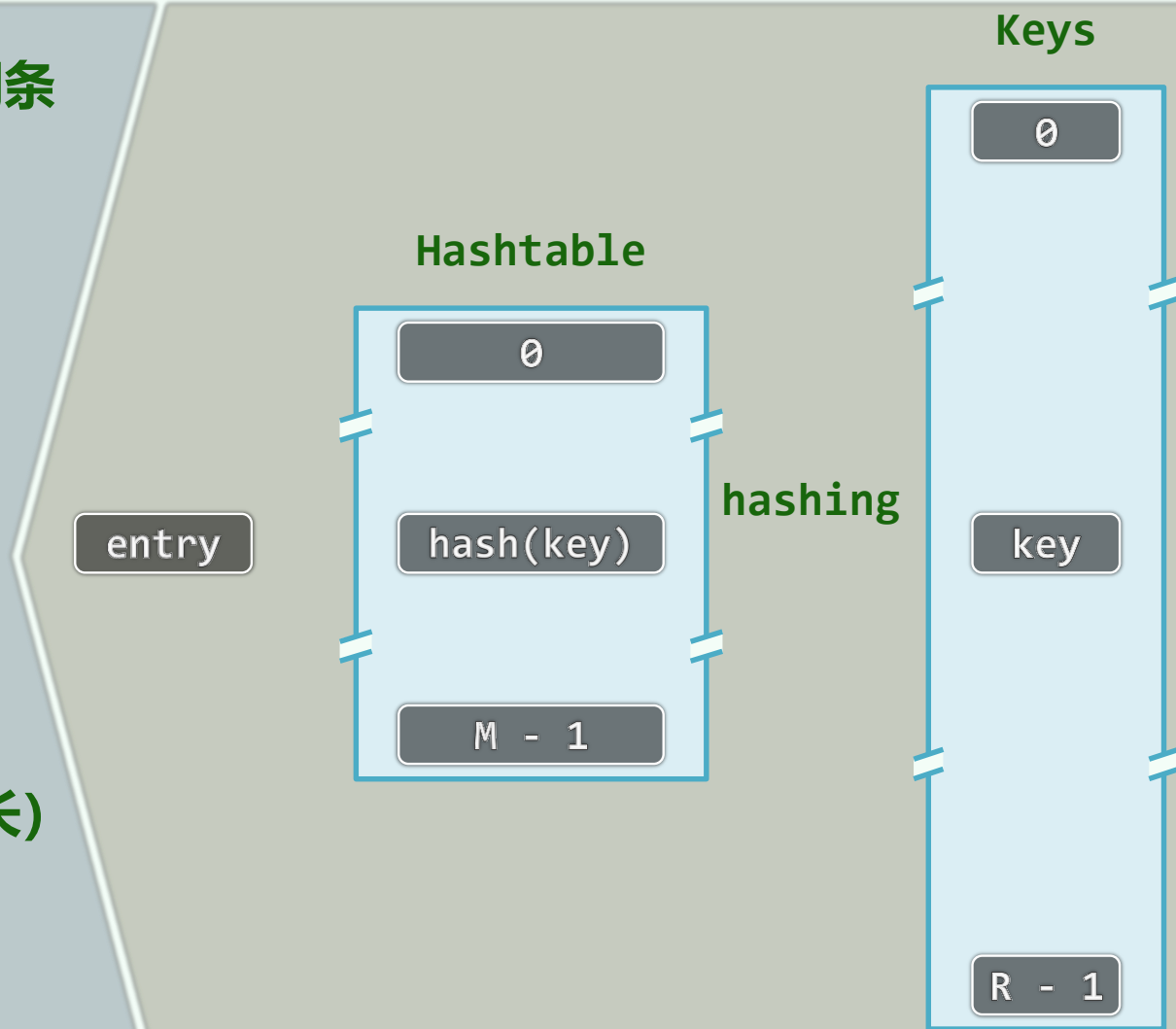
- 容量: M hashtable 容量
- 满足: $N < M \ll R$ 词条数 数组空间
- 空间: $O(N + M) = O(N)$

❖ 定址/杂凑/散列

- 根据词条的key (未必可比较)
- “直接” 确定散列表入口 (无论表有多长)

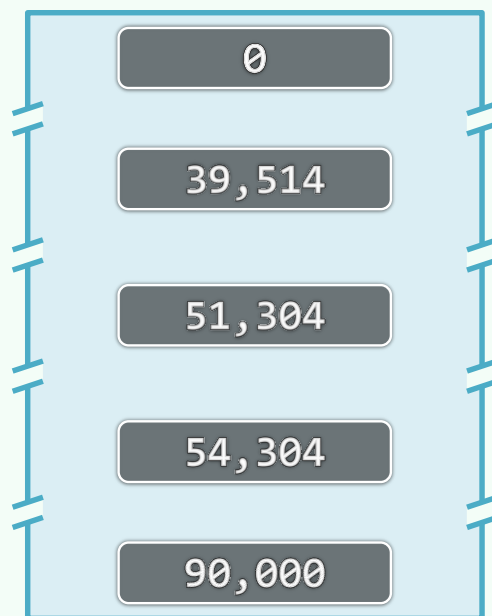
❖ 散列函数: $hash() : key \mapsto \&entry$

❖ “直接” : $expected - O(1) \neq O(1)$



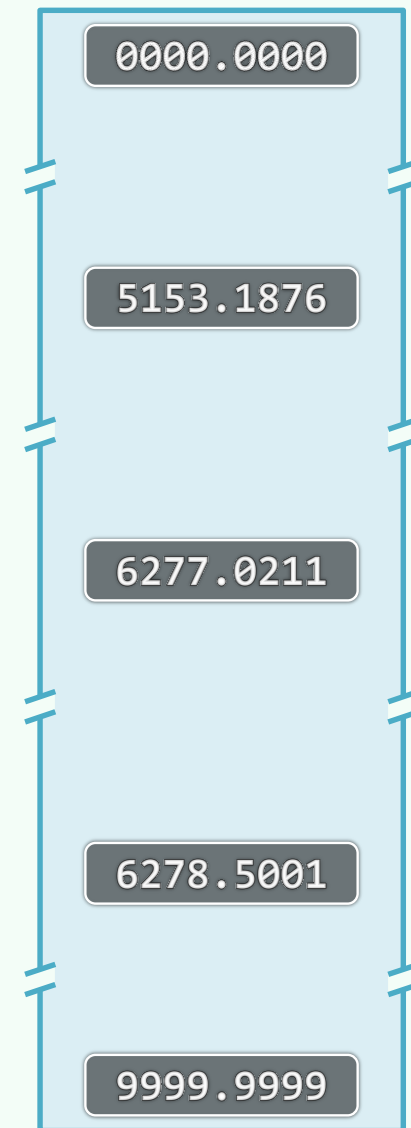
实例

校办
ZJ-9#403
查号台



$\text{hash}(\text{key}) = \text{key} \% M$

$M = 90,001$



词典

散列：冲突

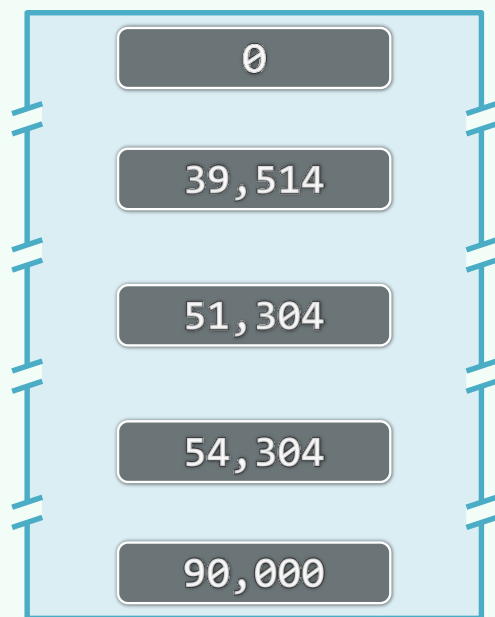
同义词

Synonym: $key_1 \neq key_2$ but $hash(key_1) = hash(key_2)$

校办

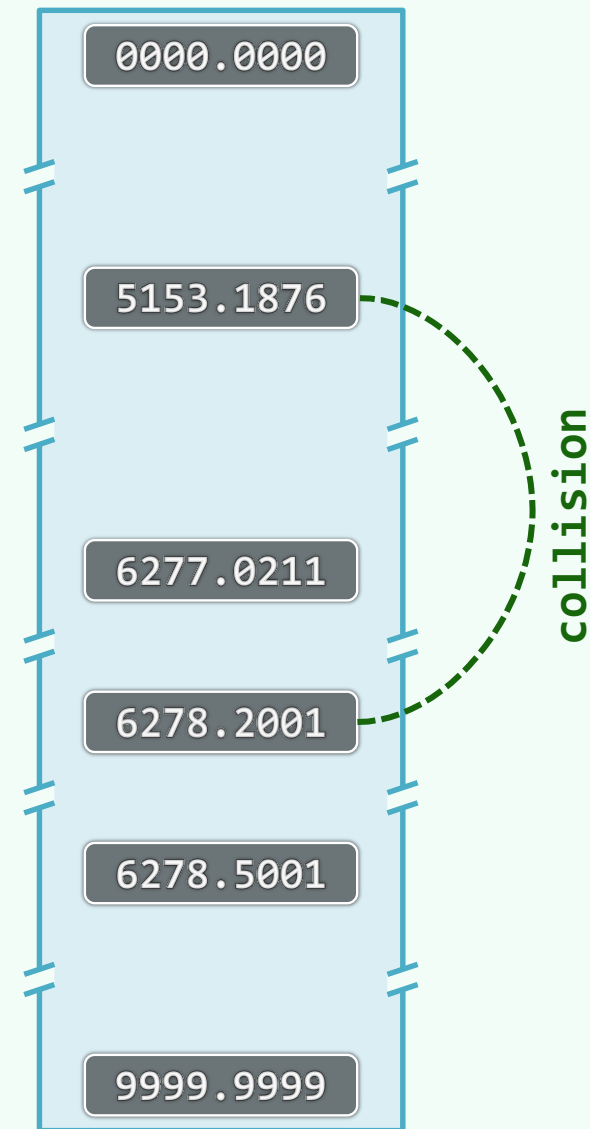
ZJ-9#403

查号台



$$M = 90,001$$

$$hash(key) = key \% M$$



装填因子 vs. 冲突

❖ load factor: $\lambda = N/M \rightarrow$ 哈希表表长

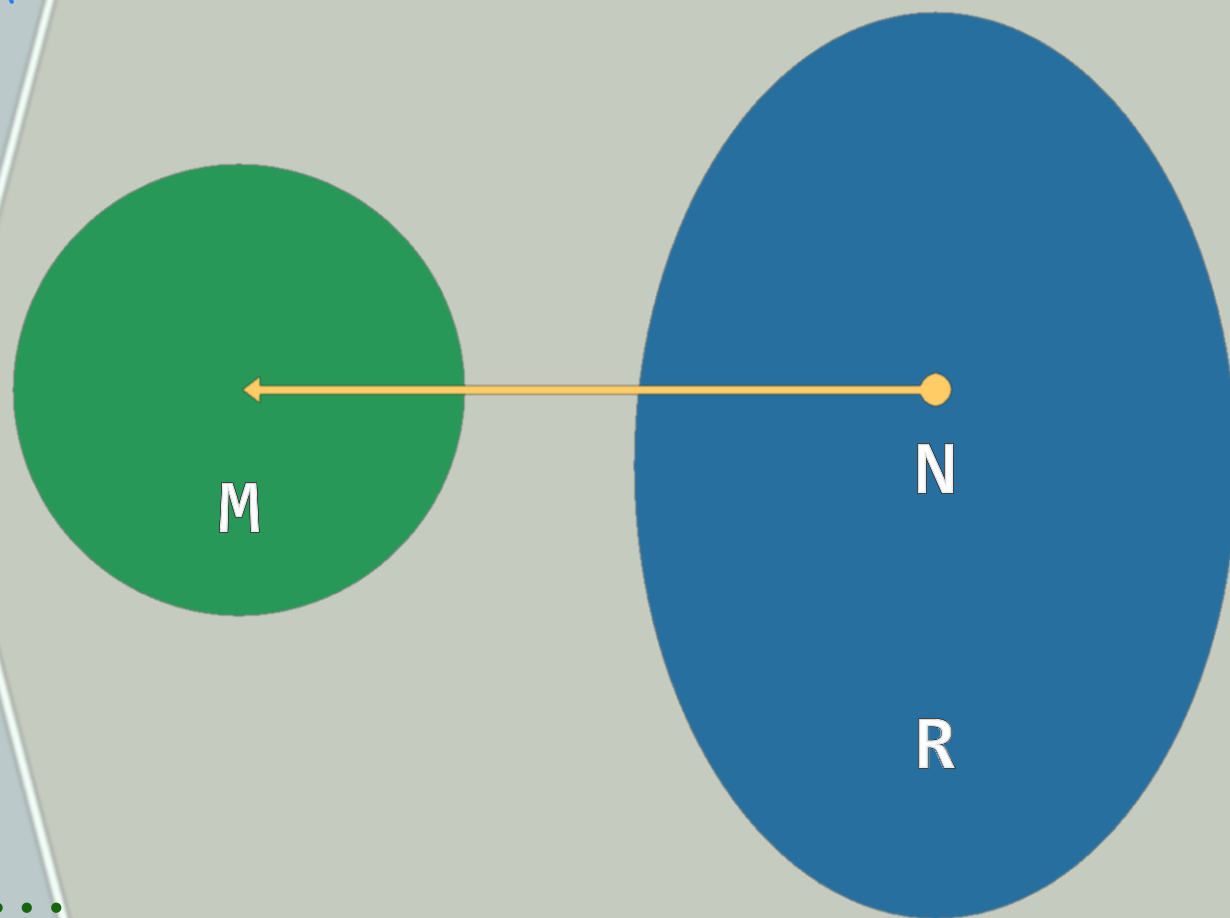
❖ λ 选多大才合适? $\downarrow$ 实际填充元素个数

❖ λ 越大/小

- 空间利用率越高/低
- 冲突的情况越严重/轻微

❖ 通过降低 λ , 冲突状况将会大大改善

但只要数据集在动态变化, 就无法彻底杜绝...



生日悖论

❖ 将在座同学（对应的词条）按生日（月/日）做散列存储

散列表长 $M = 365$ ，装填因子 = 在场人数 $N / 365$

❖ 冲突（至少有两位同学生日相同）的可能性 $P_{365}(n) = ?$

// 概率论与数理统计讲义第一章，清华大学数学系王晓峰

$$P_{365}(1) = 0, P_{365}(2) = 1/365, \dots, P_{365}(22) = 47.6\%, P_{365}(23) = 50.7\%, \dots$$

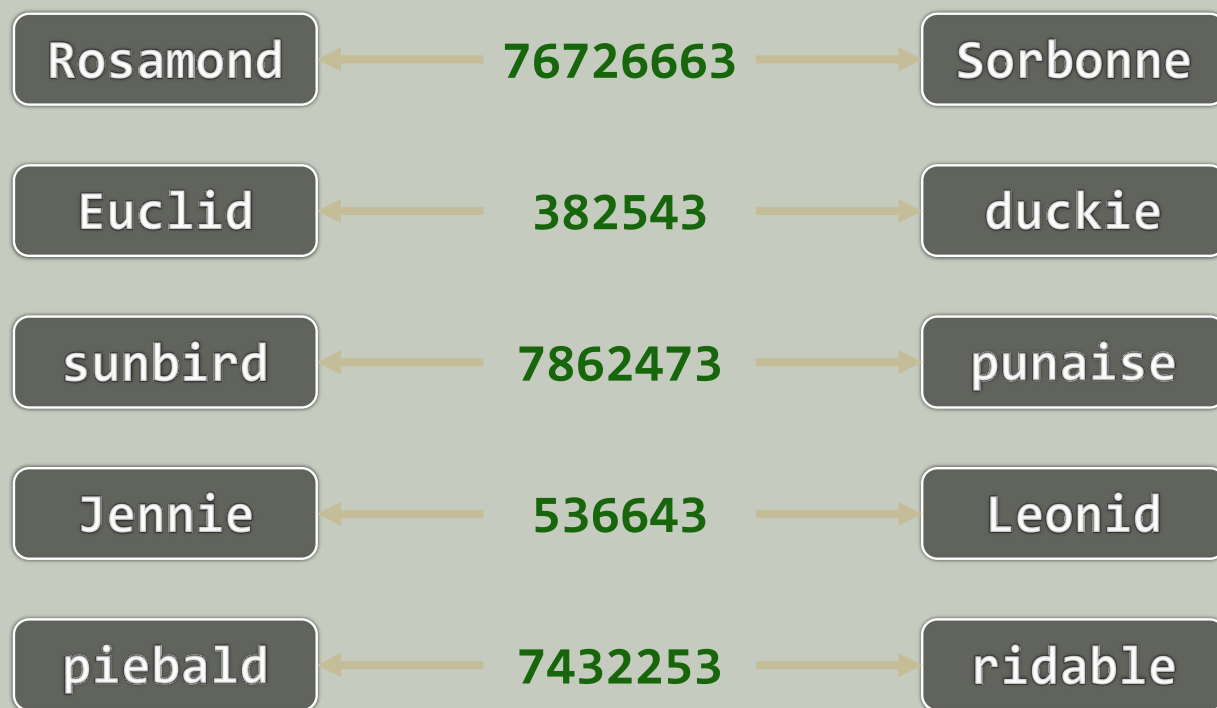
❖ 100人的集会： $1 - p_{365}(100) = 0.000,031\%$

- 自7岁起，不吃不喝、无休无息，每小时参加四次
- 到100岁，才有可能期望遇到一次无冲突的集会

❖ 因此，在装填因子确定之后，散列策略的选取将至关重要，散列函数的设计也很有讲究...

两项基本任务

- ❖ 首先（下一节）：精心**设计**散列表及散列函数，尽可能**降低**冲突的概率
- ❖ 同时（再下节）：制定可行的**预案**，以便在发生冲突时，能够尽快予以**排解**



词典
散列函数：基本

评价标准 + 设计原则

❖ 确定 (determinism)

同一关键码总是被映射至同一地址

❖ 快速 (efficiency)

expected- $O(1)$

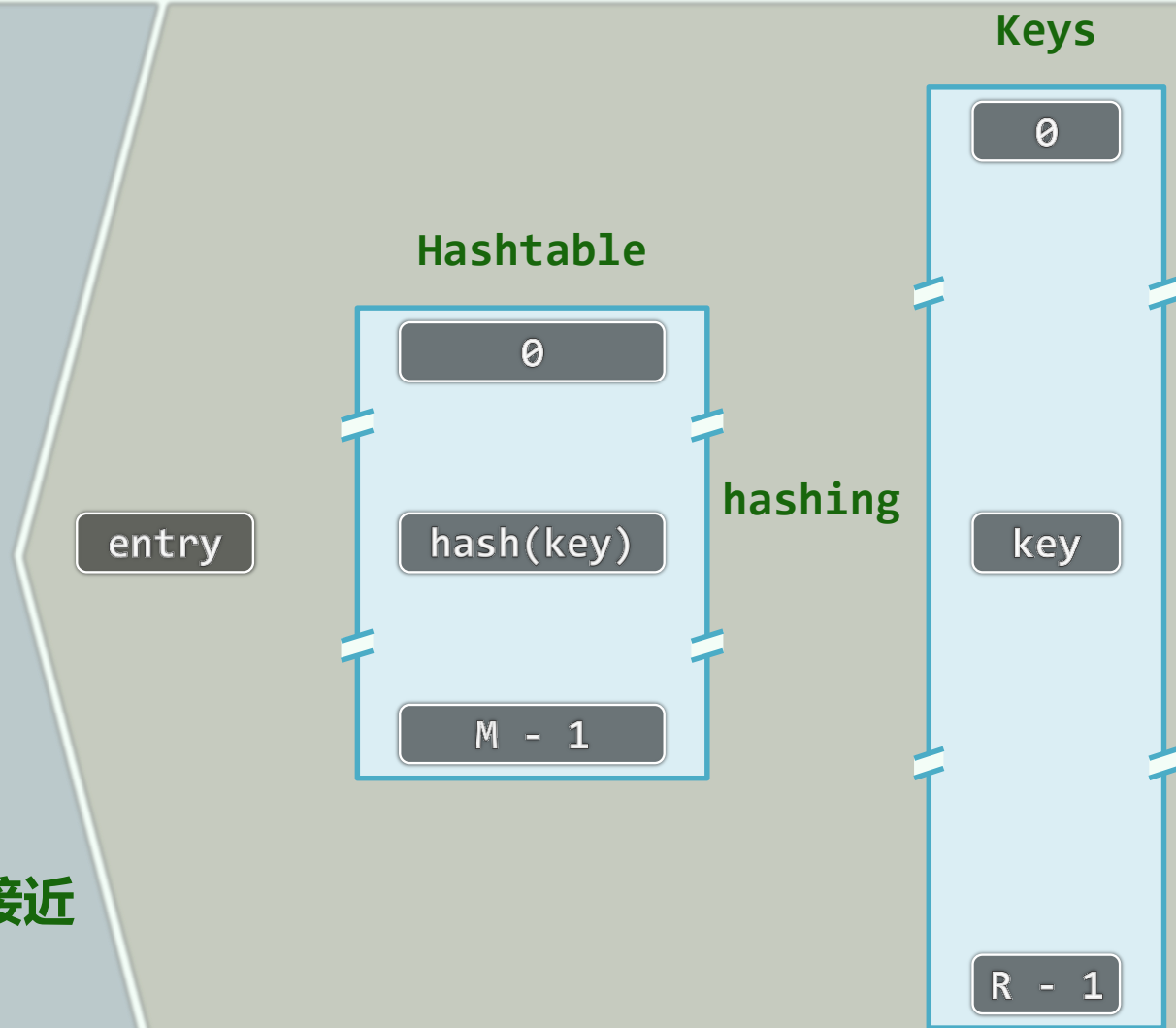
❖ 满射 (surjection)

尽可能充分地覆盖整个散列空间

❖ 均匀 (uniformity)

关键码映射到散列表各位置的**概率**尽量接近

可有效避免聚集 (clustering) 现象



除余法

❖ $hash(key) = key \% M$ //前例中，为何选 $M = 90001$?

❖ 据说： M 为素数时，数据对散列表的覆盖最充分，分布最均匀

其实：对于理想随机的序列，表长是否素数，无关紧要！

❖ 序列的Kolmogorov复杂度：生成序列的算法，最短可用多少行代码实现？

- 算术级数： 7 12 17 22 27 32 37 42 47 ... //从7开始，步长为5
- 周期级数： 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 ... //12345不断循环
- 英文： data structures and algorithms ... //频率、关联、词根、...

❖ 实际应用中的数据序列远非理想随机，上述规律性普遍存在

❖ 蝉的哲学：经长期自然选择，生命周期“取”作素数

MAD法

❖ 除余法的缺陷

$$(0+b) \% M \rightarrow (a+b+b) \% M \rightarrow \dots$$

- **不动点**: 无论表长 M 取值如何, 总有: $hash(0) \equiv 0$ 多次散列不会在一个位置

- **相关性**: $[0, R)$ 的关键码尽管系平均分配至 M 个桶; 但**相邻**关键码的散列地址也必**相邻**

增大随机性,



❖ Multiply - Add - Divide

$$hash(key) = (a \times key + b) \% M, M \text{ prime}, a > 1, b > 0, \text{ and } M \nmid a$$

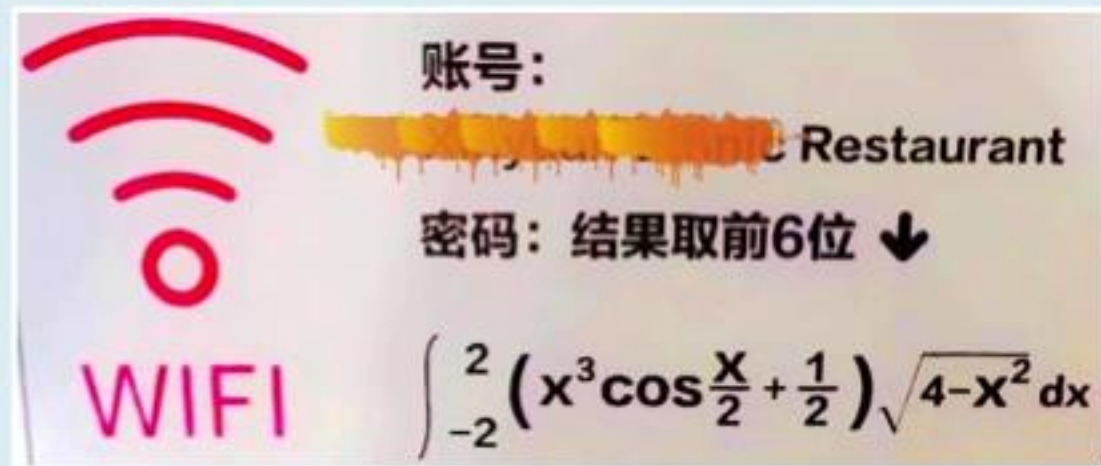
更多散列函数

❖ 数字分析 selecting digits

抽取key中的某几位, 构成地址

- 比如, 取十进制表示的奇数位

$$\text{hash}(\overset{3}{1} \overset{4}{1} \overset{5}{9} \overset{2}{6} \overset{5}{4}) = 34525$$



❖ 平方取中 mid-square

取key²的中间若干位, 构成地址

$$\text{hash}(123) = \text{middle}(123 \times 123) = 1 \boxed{512} 9 = 512$$

$$\text{hash}(1234567) = 15241 \boxed{556} 77489 = 556$$



更多散列函数

❖ 折叠法 **folding**: 将key分割成等宽的若干段, 取其总和作为地址

$$\text{- hash}(123^{456}789) = 123 + 456 + 789 = 1368 \quad // \text{自左向右}$$

$$\text{- hash}(123^{456}789) = 123 + 654 + 789 = 1566 \quad // \text{往复折返}$$

❖ 位异或法 **XOR**: 将key分割成等宽的二进制段, 经异或运算得到地址

$$\text{- hash}(110^{011}011_b) = 110 \wedge 011 \wedge 011 = 110_b \quad // \text{自左向右}$$

$$\text{- hash}(110^{011}011_b) = 110 \wedge 110 \wedge 011 = 011_b \quad // \text{往复折返}$$

❖

❖ 总之, 越是随机, 越是没有规律, 越好

词典

散列函数： hashCode与多项式法

String/Object To Integer

❖ static size_t hashCode(char s[]) {

 unsigned int h = 0;

 for (size_t n = strlen(s), i = 0; i < n; i++)

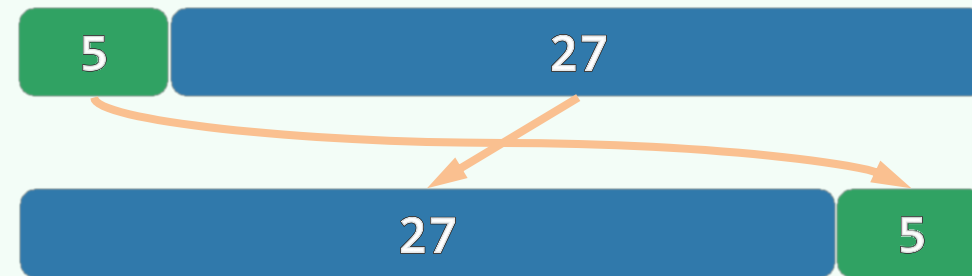
 { h = (h << 5) | (h >> 27); h += (int) s[i]; }
h × 2⁵ +

 return (size_t) h;

}

❖ 有必要如此复杂吗?

能否使用更简单的散列，比如...



$$\text{hashCode}("x_{n-1} \dots x_3 x_2 x_1 x_0")$$

$$= x_{n-1} \cdot a^{n-1} + \dots + x_2 \cdot a^2 + x_1 \cdot a^1 + x_0$$

$$= (\dots ((x_{n-1} \cdot a + x_{n-2}) \cdot a) + \dots + x_1) \cdot a + x_0$$

冲突 ~ 巧合

❖ 比如: $hashCode(S) = \sum_{c \in S} code(upper(c))$
 $hashCode("hash") = 8 + 1 + 19 + 8 = 36$

❖ 字符相对次序信息丢失, 将引发大量冲突

- I am Lord Voldemort
- Tom Marvolo Riddle

❖ 即便字符不同、数目不等...

- He's Harry Potter

❖ Key to improving your programming skills

Learning Tsinghua Data Structures & Algorithms



词典

排解冲突：开放散列

多槽位

❖ Multiple Slots

- 桶单元细分成若干槽位
- 存放（与同一单元）冲突的词条

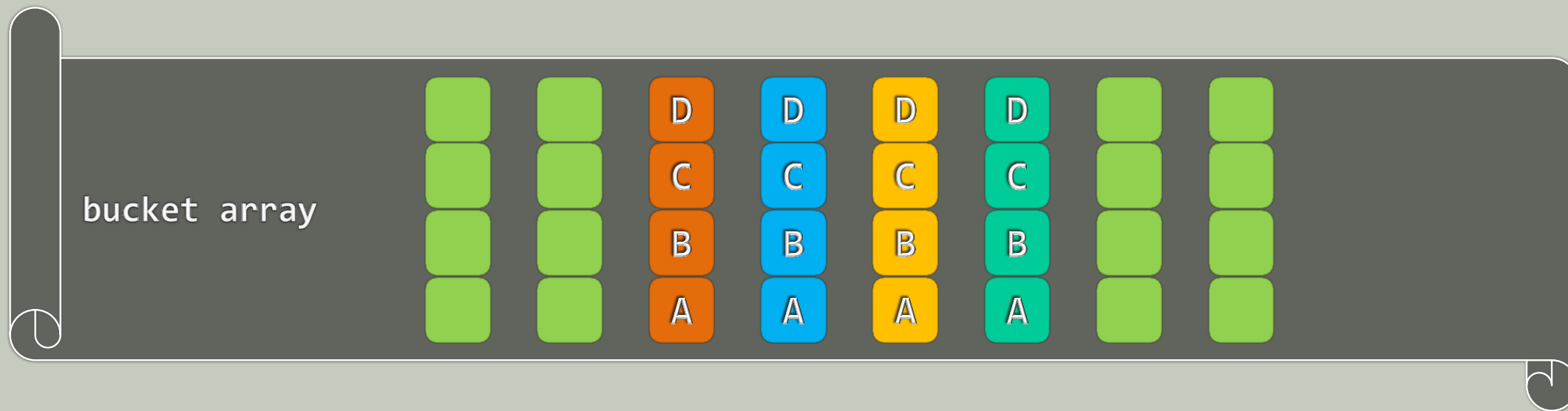
❖ 只要槽位数目不太多

依然可以保证 $O(1)$ 的时间效率

❖ 但是，究竟需要细分到什么程度？

难以预测！

- 过细，空间浪费；反过来
- 无论多细，极端情况下仍可能不够



公共溢出区

❖ Overflow Area

单独开辟一块连续空间

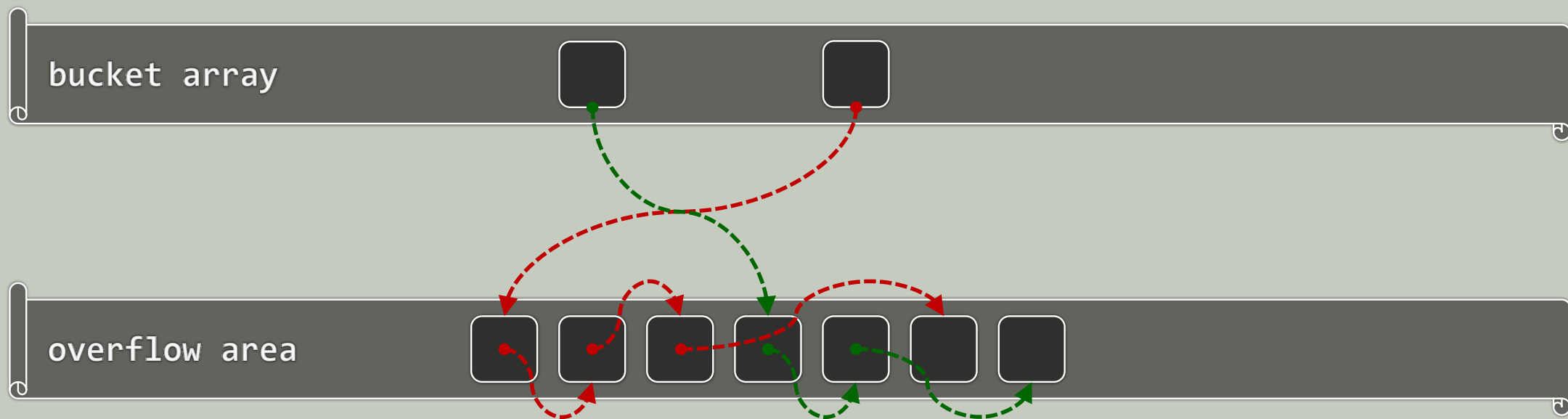
发生冲突的词条，**顺序**存入此区域

❖ 结构简单，算法易于实现

❖ 但是，不冲突则已，一旦发生冲突

最坏情况下，处理冲突词条所需的时间

正比于**溢出区**的规模



独立链

❖ Linked-List Chaining / Separate Chaining: 每个桶拥有一个列表, 存放对应的一组同义词

❖ 优点 无需为每个桶预备多个槽位

任意多次的冲突都可解决

删除操作实现简单、统一

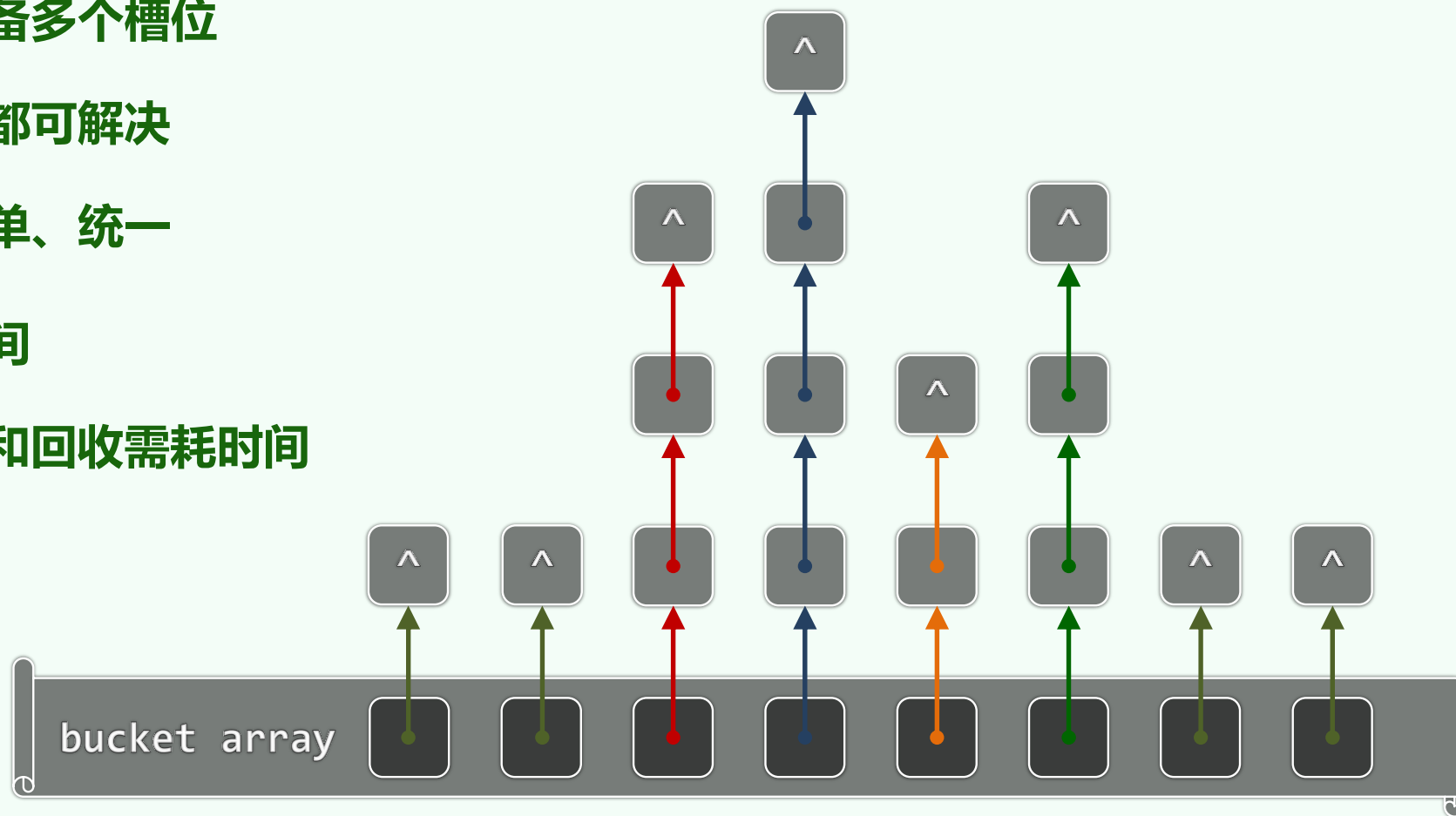
❖ 但是 指针本身占用空间

节点的动态分配和回收需耗时间

更重要的是...

❖ 空间未必连续分布

系统缓存很难生效



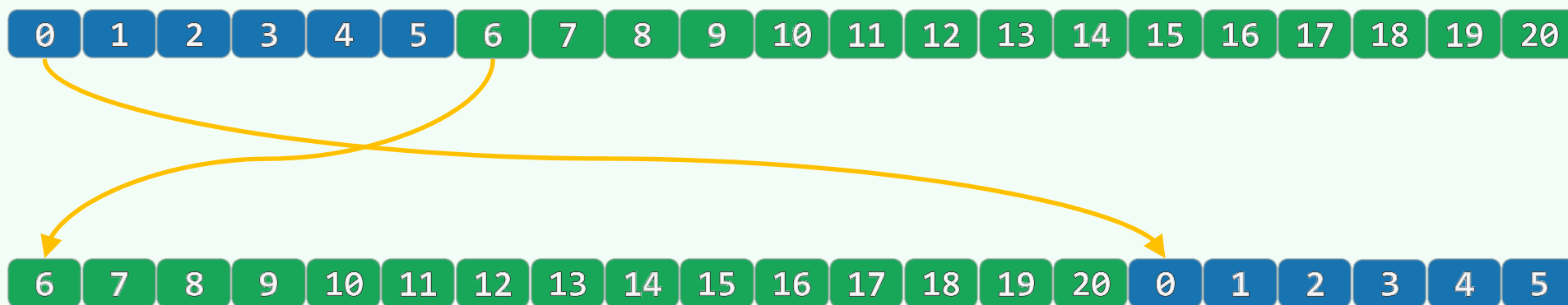
缓存

就地循环位移

❖ 仅用 $O(1)$ 辅助空间，将数组 $A[0, n)$ 中的元素向左循环移动 k 个单元

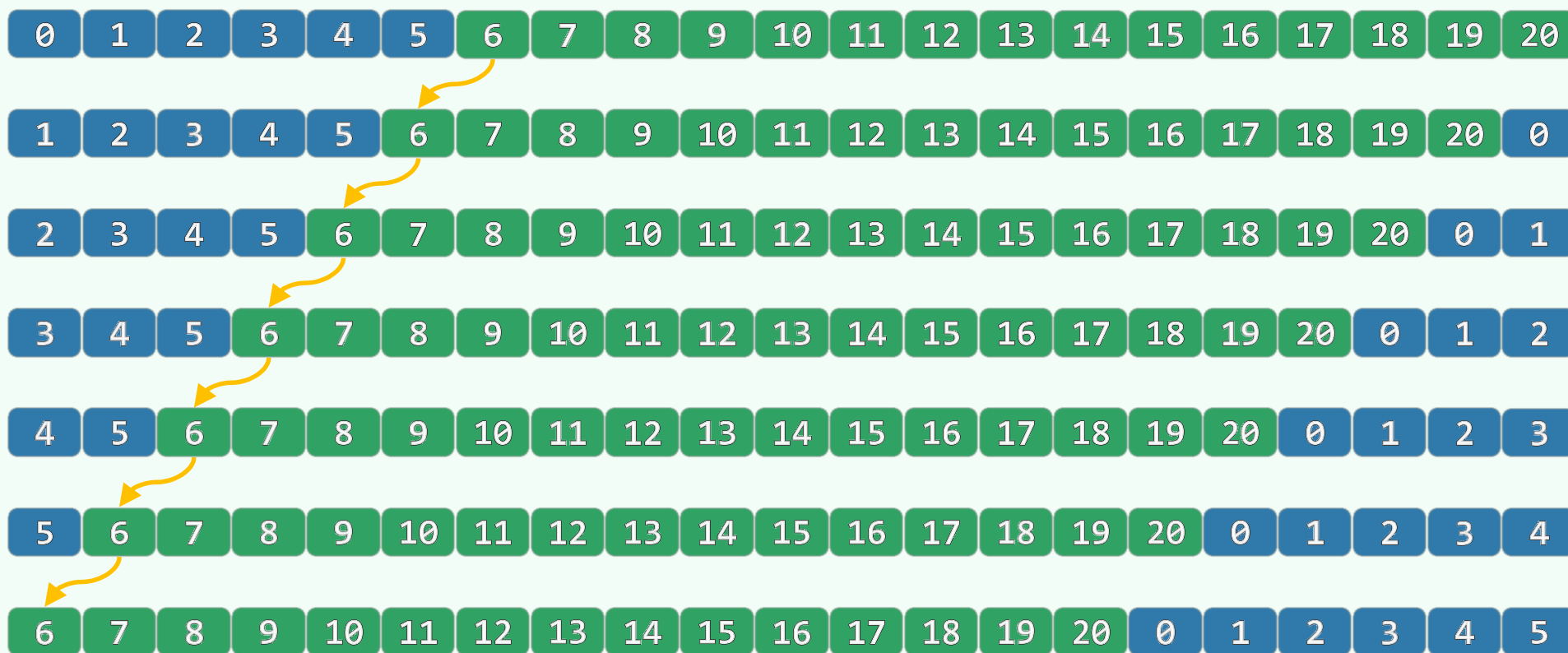
```
void shift( int * A, int n, int k );
```

❖ 比如: `shift( A, 21, 6 );`



蛮力版

❖ `void shift0( int * A, int n, int k )` //反复以1为间距循环左移 1: k O(n)
`{ while ( k-- ) shift( A, n, 0, 1 ); }` //共迭代k次, $O(n*k)$



迭代版

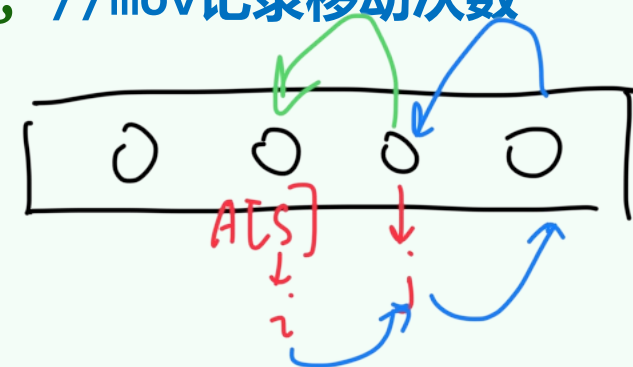
从第s个元素开始, 每个元素向左移动k.

```
int shift( int * A, int n, int s, int k ) { //  $O(n / \text{GCD}(n, k))$ 
```

```
    int b = A[s]; int i = s, j = (s + k) % n; int mov = 0; //mov记录移动次数
```

```
    while ( s != j ) //从A[s]出发, 以k为间隔, 依次左移k位
```

```
        { A[i] = A[j]; i = j; j = (j + k) % n; mov++; }
```



```
    A[i] = b; return mov + 1; //最后, 起始元素转入对应位置
```

```
    } // [0, n) 由关于k的  $g = \text{GCD}(n, k)$  个同余类组成, shift(s, k) 能够且只能够使其中之一就位
```

其它的同余类呢...



迭代版

总共n个元素, 每次移k

```
void shift1(int* A, int n, int k) { //经多轮迭代, 实现数组循环左移k位, 累计 $O(n+g)$ 
```

```
    for (int s = 0, mov = 0; mov < n; s++) // $O(g) = O(\text{GCD}(n, k))$ 
```

最少用几次就调回原状

```
        mov += shift(A, n, s, k);
```

```
}
```



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
6	1	2	9	4	5	12	7	8	15	10	11	18	13	14	0	16	17	3	19	20
6	7	2	9	10	5	12	13	8	15	16	11	18	19	14	0	1	17	3	4	20
6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	0	1	2	3	4	5

$n=21$
 $k=6$

$\text{GCD}(n, k)=3$

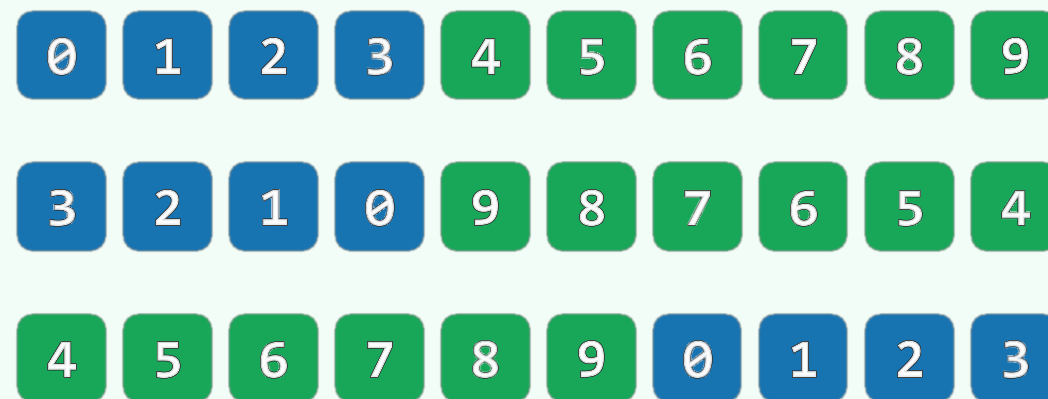
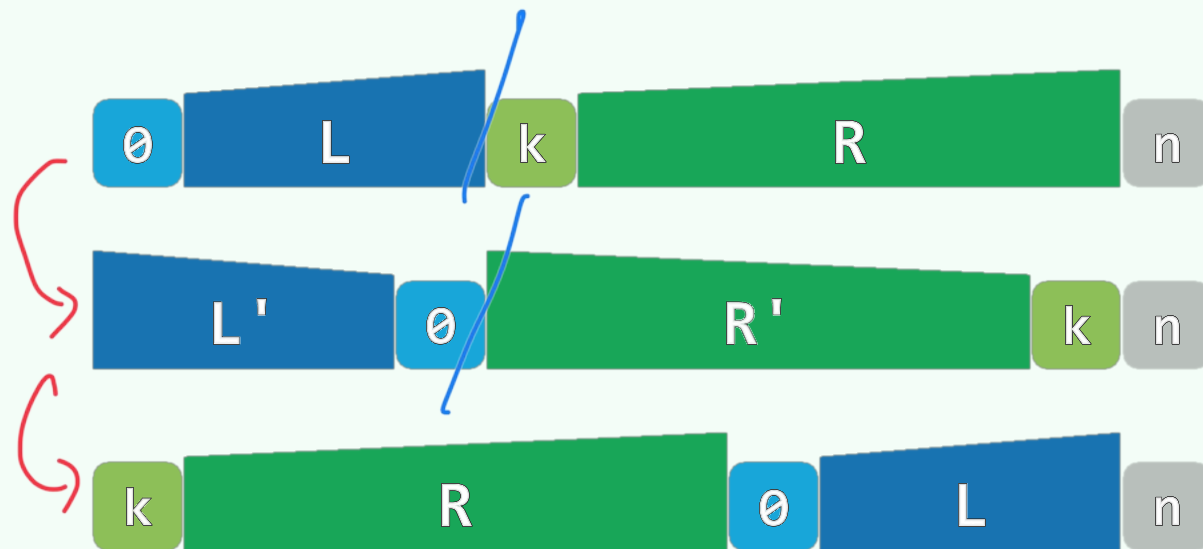
外层循环3次

每个子循环
 $\frac{n}{\text{GCD}(n, k)}$
 $= 7$ 个元素

倒置版

两次 reverse 完成 k 个数据向左移动

```
void shift2( int * A, int n, int k ) {  
  
    reverse( A, k ); //  $O(3k/2)$   
  
    reverse( A + k, n - k ); //  $O(3(n-k)/2)$   
  
    reverse( A, n ); //  $O(3n/2)$   
  
} //  $O(3n)$ 
```

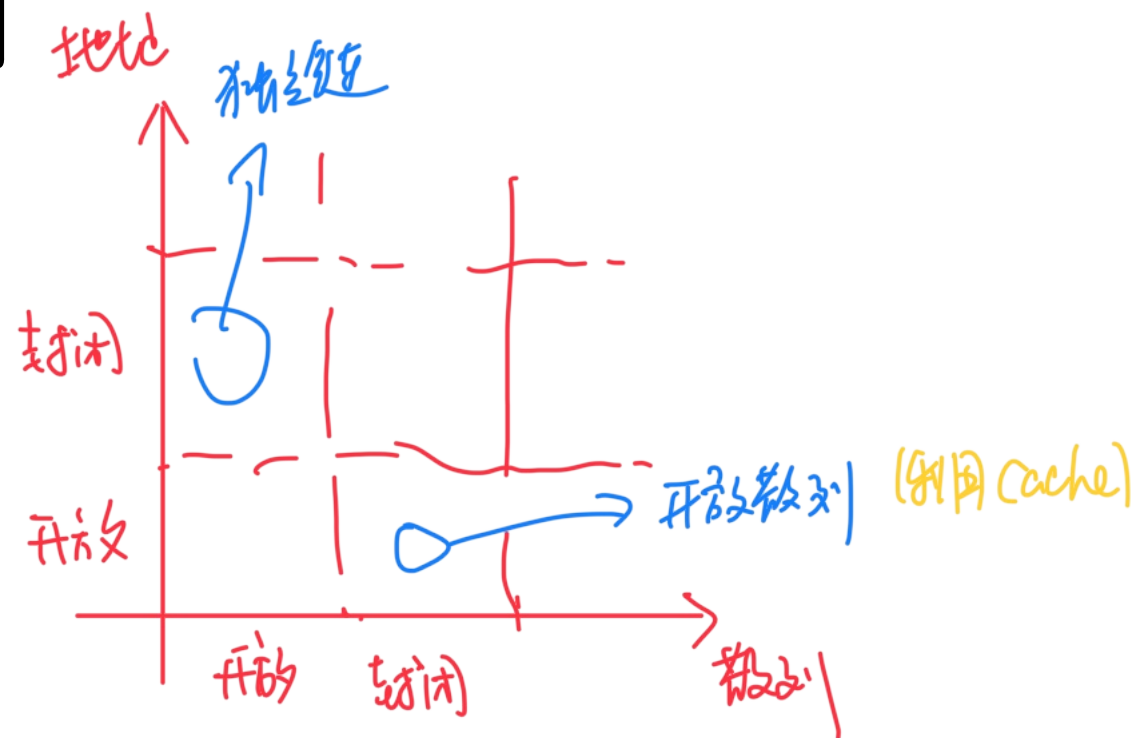


词典

排解冲突：封闭散列

封闭地址开放散列。

独立链：封闭定址法



开放定址

❖ Closed Hashing, 必然对应于Open Addressing

- 只要有必要, 任何散列桶都可以接纳任何词条

❖ Probe Sequence/Chain

- 为每个词条, 都需事先约定若干备用桶, 优先级逐次下降

❖ 查找算法: 沿试探链, 逐个转向下一桶单元, 直到

- 命中成功, 或者
- 抵达一个空桶 (存在则必能找到?) 而失败

❖ 相应地, 试探链又应如何约定?

线性试探

$$\lambda = \frac{N}{R} \quad (\text{装填因子})$$

$$\text{试探期望} = (1-\lambda) + 2\lambda(1-\lambda) + 3\lambda^2(1-\lambda) + \dots = \frac{1}{1-\lambda}$$

$$\lambda \leq 0.5 \text{ 时}$$

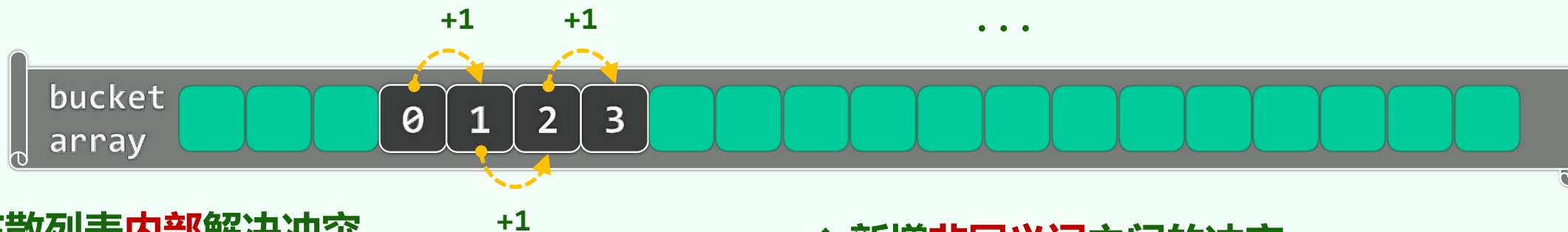
期望试探次数 ≤ 2

$$\text{空桶率} = 1 - \lambda$$

❖ Linear Probing

- 一旦冲突，则试探后一紧邻的桶
- 直到命中（成功），或抵达空桶（失败）

- ❖ $[\text{hash}(\text{key}) + 1] \% M$
- $[\text{hash}(\text{key}) + 2] \% M$
- $[\text{hash}(\text{key}) + 3] \% M$
- $[\text{hash}(\text{key}) + 4] \% M$



❖ 在散列表内部解决冲突

无需附加的指针、链表或溢出区等
整体结构保持简洁

❖ 只要还有空桶，迟早会找到

❖ 新增非同义词之间的冲突

❖ 数据堆积 (clustering) 现象严重

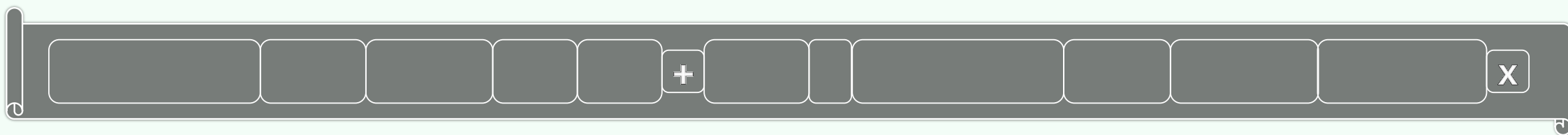
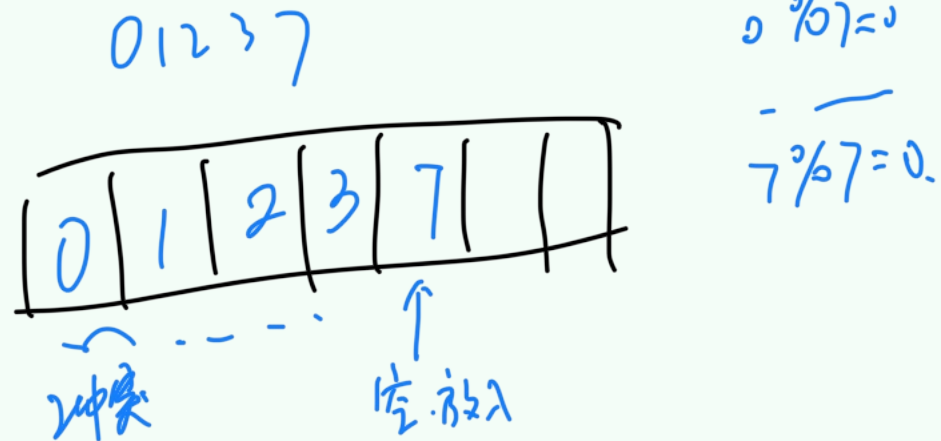
❖ 好在，试探链连续，数据局部性良好

❖ 通过装填因子，冲突与堆积都可有效控制

插入 + 删除

❖ 插入：新词条若尚不存在，则存入试探**终止**处的空桶

❖ 试探链：可能因而彼此**串接**、**重叠**！



❖ 删除：简单地清除命中的桶？

经过它的试探链都将因此**断裂**，导致后续词条**丢失**——明明存在，却访问不到

❖ 那么，如何才能**简明**、**高效**地完成删除呢？

词典

排解冲突： 懒惰删除

Lazy Removal: 故居 ~ 空宅

❖ Bitmap* removed; //用Bitmap懒惰地标记被删除的桶

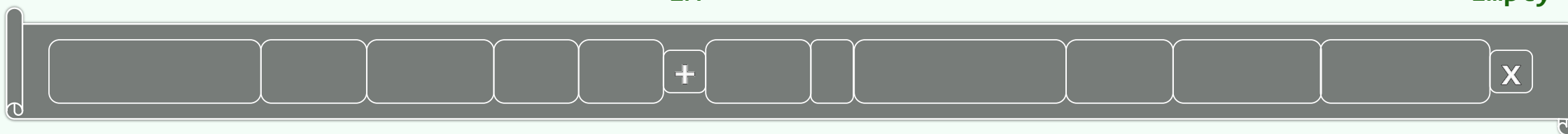
int L; //被标记桶的数目

故居

空宅

LM

Empty



❖ 仅做**标记**，不对试探链做更多调整——此后，带标记的桶，**角色**因具体的操作而异

- **查找**词条时，被视作“**必不匹配的非空桶**”，试探链在此得以**延续**
- **插入**词条时，被视作“**必然匹配的空闲桶**”，可以用来**存放**新词条

算法

```
template <typename K, typename V> int Hashtable<K, V>::probe4Hit(const K& k) {  
    int r = hashCode(k) % M; //按除余法确定试探链起点  
    while ( ( ht[r] && (k != ht[r]->key) ) || removed->test(r) )  
        r = ( r + 1 ) % M; //线性试探 (跳过带懒惰删除标记的桶)  
    return r; //调用者根据ht[r]是否为空及其内容, 即可判断查找是否成功  
}
```

不是空, hash又不一样
继续查下一个
虽然空桶
但是放到 remove 的列表中
(懒惰删除标记)
可以继续试探

```
template <typename K, typename V> int Hashtable<K, V>::probe4Free(const K& k) {  
    int r = hashCode(k) % M; //按除余法确定试探链起点  
    while ( ht[r] ) r = ( r + 1 ) % M; //线性试探, 直到空桶 (无论是否带有懒惰删除标记)  
    return r; //只要有空桶, 线性试探迟早能找到  
}
```

探测空桶失败
找到空桶放进去即可

词典

排解冲突：重散列

Rehashing

❖ template <typename K, typename V> //随着装填因子增大，冲突概率、排解难度都将激增

void Hashtable<K, V>::rehash() { //此时，不如“集体搬迁”至一个更大的散列表

int oldM = M; Entry<K, V>** oldHt = ht;

ht = new Entry<K, V>*[M = primeNLT(4 * N)]; N = 0; //新表“扩”容

memset(ht, 0, sizeof(Entry<K, V>*) * M); //初始化各桶

release(removed); removed = new Bitmap(M); L = 0; //懒惰删除标记

for (int i = 0; i < oldM; i++) //扫描原表

if (oldHt[i]) //将每个非空桶中的词条

put(oldHt[i]->key, oldHt[i]->value); //转入新表

release(oldHt); //释放——因所有词条均已转移，故只需释放桶数组本身

}

插入

❖ template <typename K, typename V>

```
bool Hashtable<K, V>::put( K k, V v ) { //散列表词条插入
```

```
    if ( ht[ probe4Hit( k ) ] ) return false; //雷同元素不必重复插入
```

```
    int r = probe4Free( k ); //为新词条找个空桶 (只要装填因子控制得当, 必然成功)
```

```
    ht[ r ] = new Entry<K, V>( k, v ); ++N; //插入
```

```
    if ( removed->test( r ) ) { removed->clear( r ); --L; } //懒惰删除标记
```

```
    if ( (N + L)*2 > M ) rehash(); //若装填因子高于50%, 重散列
```

```
    return true;
```

```
}
```

是否曾在懒惰列表

此已有元素, 不需懒惰删除标记

删除

❖ template <typename K, typename V>

bool Hashtable<K, V>::remove(K k) { //散列表词条删除算法

int r = probe4Hit(k); if (!ht[r]) return false; //确认目标词条确实存在

release(ht[r]); ht[r] = NULL; //清除目标词条 变为空

removed->set(r); --N; ++L; //更新标记、计数器 → 在probe4hit 不受阻

if (3*N < L) rehash(); //若懒惰删除标记过多, 重散列

return true;

}

词典

排解冲突：平方试探

平方试探

❖ Quadratic Probing

以平方数为距离，确定下一试探桶单元

$$[\text{hash}(\text{key}) + 1^2] \% M$$

$$[\text{hash}(\text{key}) + 2^2] \% M$$

$$[\text{hash}(\text{key}) + 3^2] \% M$$

$$[\text{hash}(\text{key}) + 4^2] \% M$$

...

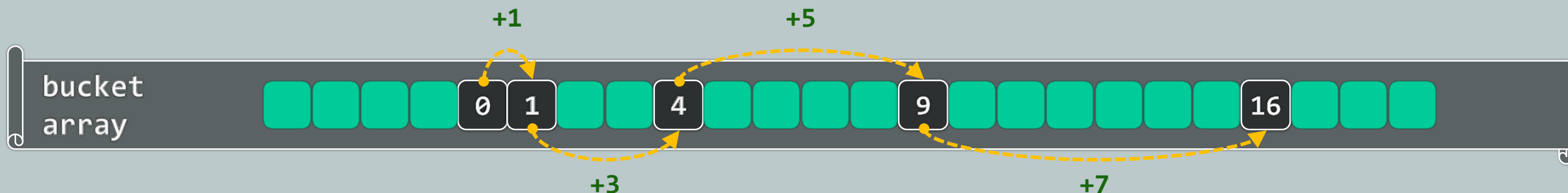
❖ 数据聚集现象有所缓解

- 试探链上，各桶间距线性递增
- 一旦冲突，可“聪明”地跳离是非之地

❖ 对于大散列表，I/O操作有所增加

❖ 只要有空桶，就...一定能...找出来吗？

//毕竟不是挨个试探

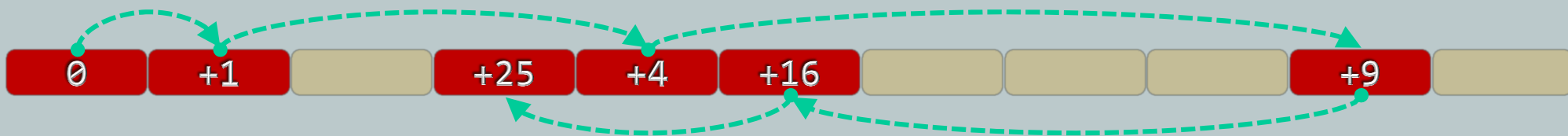
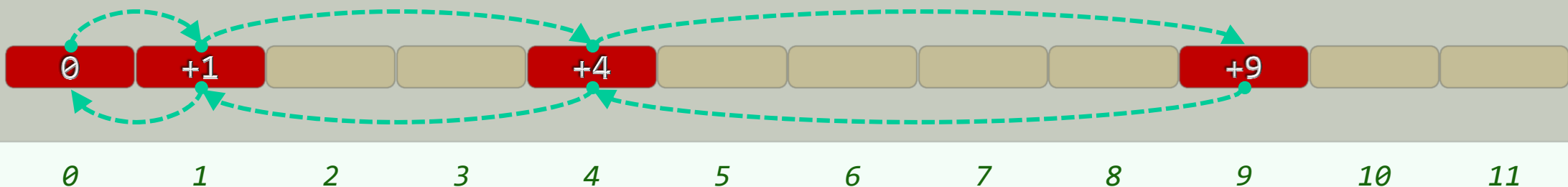


装填因子，须足够小！

④
①
⑥
(6) (2) (6)

$$\diamond \{0, 1, 2, 3, 4, 5, \dots\}^2 \% 12 = \{0, 1, 4, 9\}$$

M若为**合数**： $n^2 \% M$ 可能的取值**可能**少于 $\lceil M/2 \rceil$ 种——此时，只要对应的桶均非空...



$$\diamond \{0, 1, 2, 3, 4, 5, \dots\}^2 \% 11 = \{0, 1, 4, 9, 5, 3\}$$

M若为**素数**： $n^2 \% M$ 可能的取值**恰好**会有 $\lceil M/2 \rceil$ 种，且由试探链的前 $\lceil M/2 \rceil$ 项**取遍**

❖ **定理**：若M是素数，且 $\lambda \leq 0.5$ ，就一定能够找出；否则，不见得

试探链前缀，必足够长！

❖ 反证：假设存在 $0 \leq a < b < \lceil \mathcal{M}/2 \rceil$ ，使得

沿着试探链，第 a 项和第 b 项彼此冲突

❖ 于是： a^2 和 b^2 自然关于 $\mathcal{M}$ 同余，亦即

$$a^2 \equiv b^2 \pmod{\mathcal{M}}$$

$$b^2 - a^2 = (b + a) \cdot (b - a) \equiv 0 \pmod{\mathcal{M}}$$

❖ 然而， $0 < b - a \leq b + a < \lceil \mathcal{M}/2 \rceil + (\lceil \mathcal{M}/2 \rceil - 1) \leq \lceil \mathcal{M}/2 \rceil + \lfloor \mathcal{M}/2 \rfloor = \mathcal{M}$

无论 $b - a$ 还是 $b + a$ 都不可能整除 $\mathcal{M}$

❖ 那么，另一半的桶，可否也利用起来呢...

词典

排解冲突：双向平方试探

策略：间或地沿两个方向试探，同方向按平方确定距离

$$[\text{hash}(\text{key}) + 1^2] \% M$$

$$[\text{hash}(\text{key}) - 1^2] \% M$$

$$[\text{hash}(\text{key}) + 2^2] \% M$$

$$[\text{hash}(\text{key}) - 2^2] \% M$$

$$[\text{hash}(\text{key}) + 3^2] \% M$$

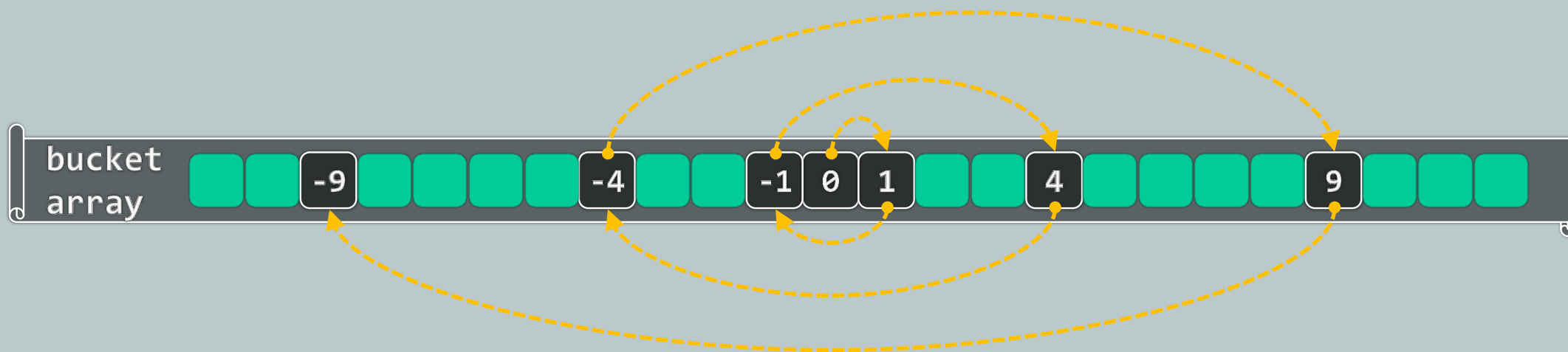
$$[\text{hash}(\text{key}) - 3^2] \% M$$

$$[\text{hash}(\text{key}) + 4^2] \% M$$

$$[\text{hash}(\text{key}) - 4^2] \% M$$

...

...



子试探链，彼此独立？

❖ **正向和反向的子试探链**，各自包含 $\lceil \mathcal{M}/2 \rceil$ 个**互异**的桶

$$-\lceil \mathcal{M}/2 \rceil, \dots, -3, -2, -1, 0, 1, 2, 3, \dots, \lfloor \mathcal{M}/2 \rfloor$$

$\pm i^2$		-36	-25	-16	-9	-4	-1	0	1	4	9	16	25	36
M	5 $4k+1$					1	4	0	1	4				
	7 $4k+3$				5 $-9\%7=5$	3	6	0	1	4	2			
	11		8	6	2	7	10	0	1	4	9	5	3	
	13	3 $-36\%13=3$	1	10	4	9	12	0	1	4	9	3	12	10

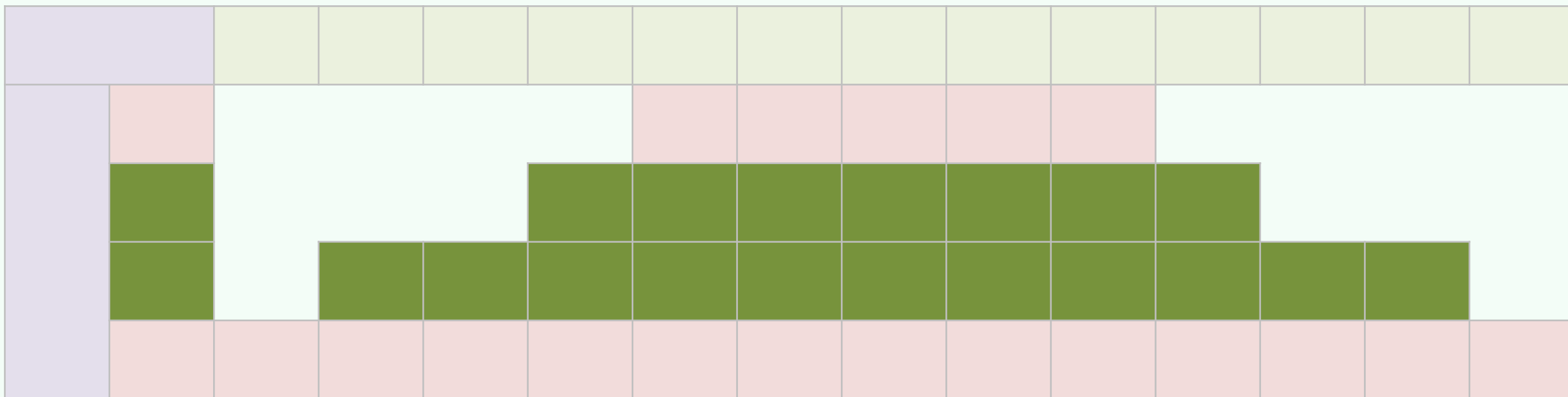
$16\%13=3$



$$4k + 3$$

❖ 两类素数: 3 5 7 11 13 17 19 23 29 31 $4k+3$
 $4k+1$

❖ 表长取作素数 $M = 4 \cdot k + 3$, 即必然可以保证试探链的前 M 项均互异



❖ 反之, $M = 4 \cdot k + 1$ 就...必然不能使用?

Two-Square Theorem of Fermat

❖ 任一素数 p 可表示为一对整数的平方和，当且仅当

$$p \equiv 1 \pmod{4}$$

❖ 只要注意到：

$$\begin{aligned}(u^2 + v^2) \cdot (s^2 + t^2) &= (us + vt)^2 + (ut - vs)^2 \\ &= (us - vt)^2 + (ut + vs)^2\end{aligned}$$

$$\begin{aligned}(2^2 + 3^2) \cdot (5^2 + 8^2) &= (10 + 24)^2 + (16 - 15)^2 \\ &= (10 - 24)^2 + (16 + 15)^2\end{aligned}$$

❖ 就不难推知：

- 自然数 n 可表示为一对整数的平方和，当（且仅当）
- 它的每一 $M = 4 \cdot k + 3$ 类的素因子均为偶数次方

$$\begin{aligned}1 &= 1^2 + 0^2 \\ 2 &= 1^2 + 1^2 \\ 3 &= \\ 4 &= 2^2 + 0^2 \\ 5 &= 2^2 + 1^2 \\ 6 &= \\ 7 &= \\ 8 &= 2^2 + 2^2 \\ 9 &= 3^2 + \\ 10 &= 3^2 + \\ 11 &= \\ &\vdots\end{aligned}$$



$$2 = (1^2 + 1^2)$$

$4k+1$ 型有如此处理

$4k+3$ 素数为偶数次方

$$(p^2 + 0^2)$$

当表长为 $4K+3$ 时，查找链前 M 步都不会发生冲突。

(平方试探)

反证：假设在正向试探中的第 a 步与逆向试探中的第 b 步是冲突的。 $1 \leq b, a \leq \lfloor M/2 \rfloor$ ，第0步是公共的。

冲突 $\Rightarrow$

M 是素数

$$-b^2 \equiv a^2 \pmod{M} \Rightarrow \text{移项}$$

$$a^2 + b^2 = n \equiv 0 \pmod{M} \Rightarrow$$

M 必然是 n 的素因子之一。

根据费马双平方定理， M 至少是平方项 $\Rightarrow$

$$a^2 + b^2 \geq M^2 \text{ 与 } b, a \leq \lfloor M/2 \rfloor \text{ 相矛盾。}$$

M 只是一个因子

词典

排解冲突：再散列

Double Hashing

❖ 预先约定第二散列函数: $hash_2()$ $[hash(key) + 1 \times hash_2(key)] \% \mathcal{M}$

❖ 一旦发生冲突 $[hash(key) + 2 \times hash_2(key)] \% \mathcal{M}$

以 $hash_2(key)$ 为偏移增量, 重新确定地址 $[hash(key) + 3 \times hash_2(key)] \% \mathcal{M}$

❖ 比如, $hash_2(key) \equiv 1$ 时, 即是线性试探 $\dots\dots$

